```
class RecurrenceRelation {
    /**
    * Definition: An equation or inequality that describes
    * a function in terms of its value on smaller inputs.
    */

    const concept = "Self-Similarity";
    let usage = ["Algorithm Analysis", "Time Complexity"];

    // Analogy: Russian Nesting Dolls
```

```
}
```

Why are Recurrences Important?

// Foundation of Divide-and-Conquer Analysis

1. Divide

Break the problem into smaller subproblems (instances of the same problem).

2. Conquer

Recursively solve subproblems. If small enough, solve directly (Base Case).

3. Combine

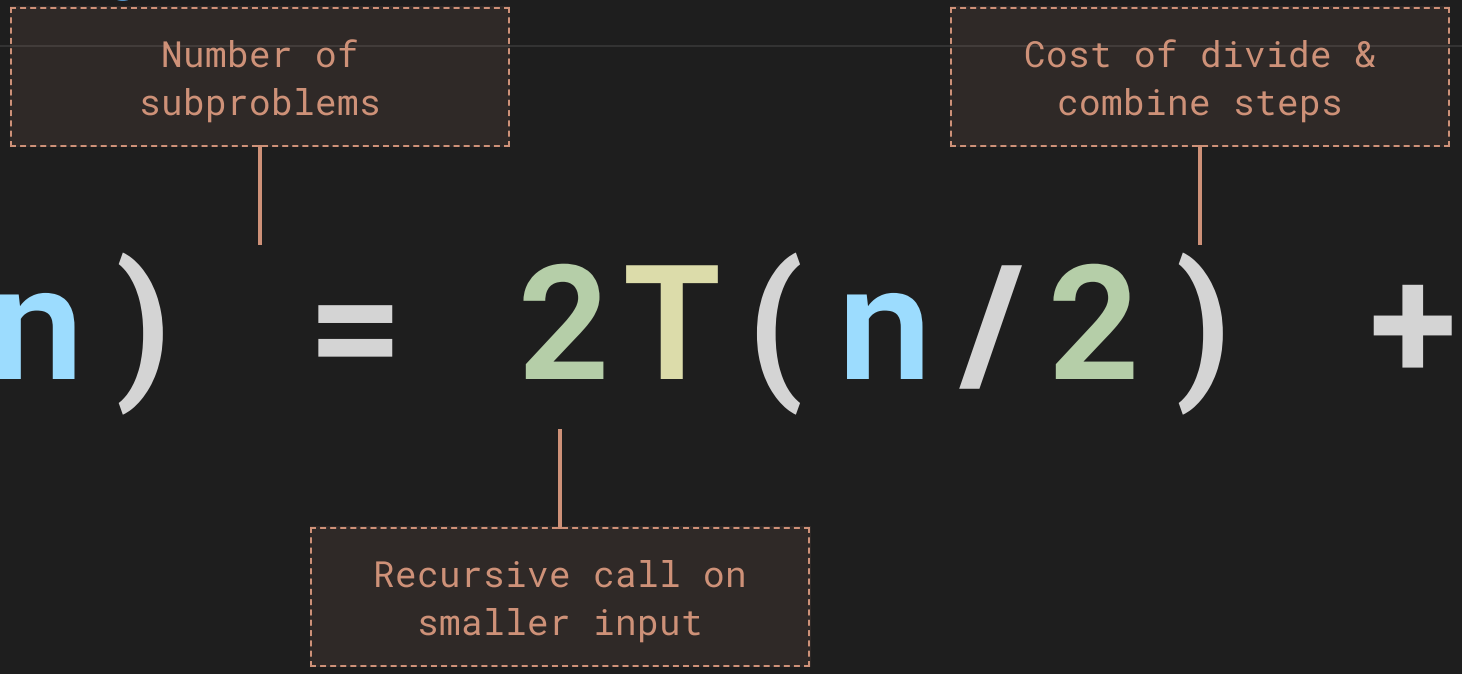
Merge the solutions of subproblems to solve the original problem.

Example: Merge Sort Running Time

$$T(n) = 2T(n/2) + O(n)$$

// 2 subproblems of size n/2 + linear time to merge

The Anatomy of a Recurrence



The Base Case

The condition that stops the recursion. Typically for small n (e.g., $n=1$), the problem is solved directly in $O(1)$ time.

Methods for Solving Recurrences

```
function substitution() {
  step1 = "Guess";
  step2 = "Verify (Induction)";
}
```

The "Guess & Prove" Approach

```
// Most general method.
// Requires a good initial guess.
// Used to prove O and Ω bounds.
```

```
function recursionTree() {
  visualize("Tree");
  return sum(costs_per_level);
}
```

The Visual Approach

```
// Models the flow of recursion.
// Sums costs at each depth.
// Great for generating guesses.
```

```
function masterMethod() {
  if (form == "aT(n/b) + f(n)")
    return lookup(Case);
}
```

The "Cookbook" Approach

```
// Quickest for standard forms.
// Relies on 3 specific cases.
// Compares n^(log_b a) vs f(n).
```

Substitution Method

// The "Guess and Prove" Technique

function make_guess()

Characterize the form of the solution based on experience or heuristics (e.g., Recursion Tree).

guess = "O(n log n)"

function verify_induction()

Use `Mathematical Induction` to find the constants c and n_0 and prove the guess is correct.

Show that $T(n) \leq c \cdot f(n)$.

⚠ **Warning:** This method is powerful but requires a good initial guess. If the guess is wrong, the induction will fail.

Substitution Method: Simple Example

// Solving T(n) = T(n-1) + 1

Recurrence: $T(n) = T(n-1) + 1$ Base Case: $T(1) = 1$

Step 1: Guess

We guess $T(n) = O(n)$

Goal: Prove $T(n) \leq cn$

Step 2: Inductive Step

Assume $T(n-1) \leq c(n-1)$

$T(n) = \mathbf{T(n-1)} + 1$

$T(n) \leq \mathbf{c(n-1)} + 1$

$T(n) = cn - c + 1$

We need: $cn - c + 1 \leq cn$

$$\mathbf{c \geq 1}$$

Step 3: Base Case

For $n = 1$:

$T(1) = 1$

True if $c \geq 1$

✅ Proof holds for $c \geq 1$. Thus, $T(n) = O(n)$.

Substitution: Complex Example

Target: $T(n) = 2T(n/2) + n$

1. The Guess

$T(n) = O(n \lg n)$

2. Inductive Hypothesis

Assume for $k < n$:

$T(k) \leq c \cdot k \lg k$

Specifically for $n/2$:

$T(n/2) \leq c(n/2)\lg(n/2)$

✓ Proven for $c \geq 1$

$$
\begin{aligned}
&T(n) = 2T(n/2) + n && \text{// Original Recurrence} \\
&\leq 2[c(n/2)\lg(n/2)] + n && \text{// Substitute Hypothesis} \\
&= cn(\lg n - \lg 2) + n && \text{// Simplify: } \lg(a/b) = \lg a - \lg b \\
&= cn \lg n - cn + n && \text{// Expand terms } (\lg 2 = 1) \\
&= cn \lg n - (c - 1)n && \text{// Group residual terms} \\
&\leq cn \lg n && \text{// True if } (c - 1)n \geq 0 \Rightarrow c \geq 1
\end{aligned}
$$

Recursion-Tree Method

// Visualizing the Cost

1. Visualize

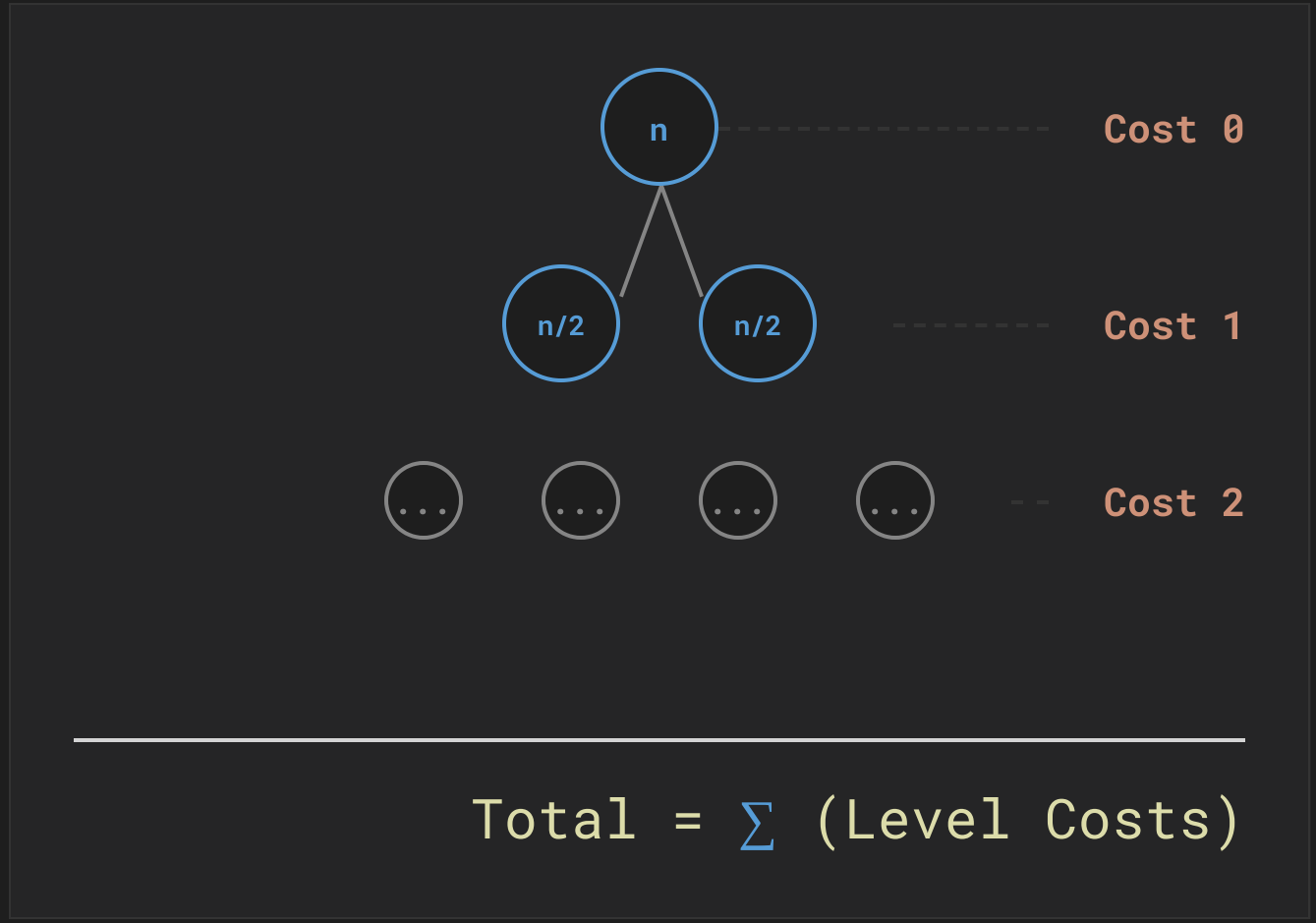
Draw a tree where each node represents the cost of a single subproblem.

2. Sum Levels

Sum the costs of all nodes at each depth level to get per-level costs.

3. Total

Sum all per-level costs to determine the total running time.

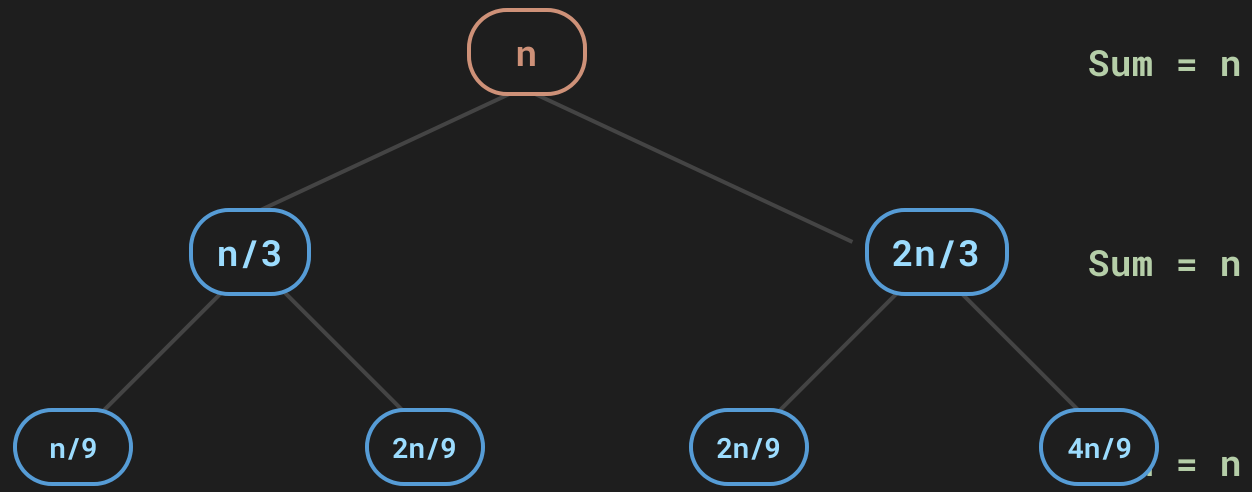


Recursion-Tree: Simple Example

$$T(n) = 2T(n/2) + n^2$$

Recursion Tree: Complex Example

$$T(n) = T(n/3) + T(2n/3) + n$$



Shortest Path: $\log_3 n$

Longest Path: $\log_{3/2} n$

1. Unbalanced Tree

The tree leans to the right. The $2n/3$ branch reduces the problem size slower than the $n/3$ branch, creating a deeper tree on the right.

2. Cost Per Level

At every level (until leaves appear), the cost sums to exactly n. (e.g., $n/3 + 2n/3 = n$)

3. Total Cost

Total ≈ (Cost per level) × (Number of levels) $O(n \log n)$

Master Method: Introduction

// The "Cookbook" Approach

Standard Form

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive

$f(n)$

Cost of the work done at the **root**
(divide & combine)

VS

$n^{\log_b a}$

Number of leaves in the recursion tree
(Watershed Function)

> **The Comparison Principle:** The solution is determined by whichever function grows faster.

Master Method: Case 1

// The "Leaves Dominate" Case

if $f(n) = O(n^{\log_b a - \varepsilon})$ // for $\varepsilon > 0$

Leaves > Root

then $T(n) = \Theta(n^{\log_b a})$

Simple Example

$T(n) = 8T(n/2) + n^2$

$a=8$, $b=2$

Watershed: $n^{\log_2 8} = n^3$

Driving: $f(n) = n^2$

$n^2 < n^3$ (by factor n^1)

⇒ **$T(n) = \Theta(n^3)$**

Complex Example

$T(n) = 9T(n/3) + n$

$a=9$, $b=3$

Watershed: $n^{\log_3 9} = n^2$

Driving: $f(n) = n$

$n < n^2$ (by factor n^1)

⇒ **$T(n) = \Theta(n^2)$**

Master Method: Case 2

Balanced Growth

$$f(n) = \Theta(n^{\log_b a})$$

The driving function $f(n)$ grows at the **same rate** as the watershed function.

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

Multiply the watershed function by a logarithmic factor.

Simple: Merge Sort

$$T(n) = 2T(n/2) + n$$

$a = 2 \quad b = 2 \quad f(n) = n$

$$n^{\log_2 2} = n^1 = f(n)$$

$$\Theta(n \lg n)$$

Complex: Binary Search

$$T(n) = T(n/2) + 1$$

$a = 1 \quad b = 2 \quad f(n) = 1$

$$n^{\log_2 1} = n^0 = 1 = f(n)$$

$$\Theta(\lg n)$$

REQUIRED CHECK

Master Method: Case 3

// The "Root Dominates" Case

PRIMARY CONDITION

$$f(n) = \Omega(n^{\log_b a + \varepsilon})$$

Driving grows polynomially faster than leaves.

then $T(n) = \Theta(f(n))$

Regularity Condition:

$af(n/b) \leq cf(n)$

for some constant $c < 1$ and large n.

Simple

$$T(n) = 2T(n/2) + n^3$$

Watershed:	n^1	
Driving:	n^3	
Compare:	$n^3 > n^1$ ($\varepsilon=2$)	
Regularity:	$2(n/2)^3 = n^3/4 \leq cn^3$	

$\Theta(n^3)$

Complex

$$T(n) = 3T(n/4) + n \lg n$$

Watershed:	$n^{\log_4 3}$	$\approx$	$n^{0.79}$	
Driving:	n	$\lg$	n	
Compare:	$n^1 > n^{0.79}$ ($\varepsilon\approx0.2$)			
Regularity:	$3(n/4 \lg(n/4)) \leq cn \lg n$			

$\Theta(n \lg n)$

Summary & Practice

// Recap and Challenge Problems

Solutions to Practice Problems

// Master Method Application

Problem 1 — CASE 1

$$T(n) = 4T(n/2) + n$$

$a = 4,\ b = 2$
$f(n) = n$

$n^{\log_2 4} = n^2$

$n < n^2$
(Leaves Dominate)

$\Theta(n^2)$

Problem 2 — CASE 2

$$T(n) = 4T(n/2) + n^2$$

$a = 4,\ b = 2$
$f(n) = n^2$

$n^{\log_2 4} = n^2$

$n^2 = n^2$
(Balanced)

$\Theta(n^2 \lg n)$

Problem 3 — CASE 3

$$T(n) = 4T(n/2) + n^3$$

$a = 4,\ b = 2$
$f(n) = n^3$

$n^{\log_2 4} = n^2$

$n^3 > n^2$
(Root Dominates)

$\Theta(n^3)$